

Jenkins

El orquestador que ejecuta tu pipeline de seguridad.
De cero a un pipeline DevSecOps funcionando, en 90 minutos.

Jenkins LTS

Pipeline as Code

Gitleaks

SonarQube

Trivy

OWASP ZAP

DefectDojo

Agenda

Tiempo	Bloque
0:00 – 0:10	De dónde venimos: tu pipeline en GitLab CI
0:10 – 0:30	Qué es Jenkins y cómo piensa (teoría útil)
0:30 – 0:40	Demo en vivo: primer arranque y primer job
0:40 – 1:20	PRÁCTICA: 3 niveles, de "Hola mundo" al pipeline completo
1:20 – 1:30	Resultados, comparativa y cierre

Al salir de aquí sabrás...

- Qué es un **orquestador CI/CD** y por qué Jenkins sigue mandando en la empresa.
- Crear **jobs** y escribir un **Jenkinsfile** declarativo.
- Guardar tokens en **Credentials** y usarlos sin exponerlos.
- Lanzar **herramientas de seguridad en contenedores** desde el pipeline.
- Convertir un aviso en un **quality gate que bloquea**.

No necesitas saber Java. Jenkins está escrito en Java, pero tú vas a escribir un fichero de texto de 30 líneas y a pulsar botones.

BLOQUE 1

De dónde venimos

Lo que ya construiste con GitLab CI, y la pregunta que deja abierta.

Ya montaste esto: un pipeline DevSecOps



La idea que sostiene todo: *Shift-Left*

Mover los controles de seguridad lo más **temprano** posible: detectar en el `git push`, no en producción tres meses después.

Un fallo detectado en el IDE cuesta **x1**. En producción, entre **x30 y x100**.

Lo importante: las herramientas (Gitleaks, Trivy, ZAP...) **no son** GitLab. Son programas independientes. GitLab CI solo era **quien las llamaba**.

¿Quién ejecuta todo eso, y cuándo?

Un **orquestador de CI/CD** es el programa que:

- **Escucha** un evento (un push, una hora, un botón).
- **Lee** un fichero con la receta (el pipeline).
- **Ejecuta** cada paso en el orden correcto, en una máquina limpia.
- **Guarda** logs, informes y artefactos.
- **Decide** si el resultado es verde, amarillo o rojo.

GitLab CI es un orquestador que viene *dentro* de GitLab.

Jenkins es un orquestador **independiente**: no le importa dónde esté tu código.

Otros orquestadores que verás

Jenkins

GitLab CI

GitHub Actions

Azure DevOps

TeamCity

CircleCI

Bamboo

Argo / Tekton

Cambia la sintaxis. **No cambian los conceptos**: etapas, artefactos, credenciales, agentes, quality gates.

BLOQUE 2

Qué es Jenkins

Historia mínima, arquitectura y el vocabulario que vas a usar toda la práctica.

Jenkins en una frase

Jenkins es un **servidor de automatización** de código abierto: un programa que ejecuta tareas (*jobs*) de forma automática cuando ocurre algo, y te enseña el resultado.

Historia (30 s)

- **2004** — nace como *Hudson* en Sun Microsystems.
- **2010** — Oracle compra Sun y se queda la marca *Hudson*.
- **2011** — la comunidad hace *fork* y lo llama **Jenkins**.
- **2016** — Jenkins 2.0: llega el **Pipeline as Code** (el [Jenkinsfile](#)).
- **Hoy** — proyecto de la Continuous Delivery Foundation, **+1.800 plugins**.

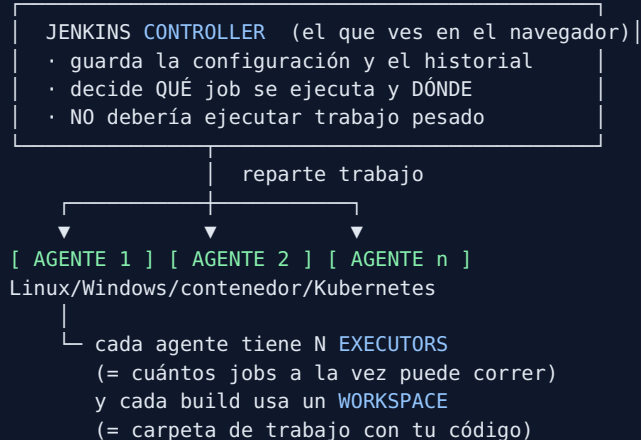
Por qué te lo vas a encontrar sí o sí

- Es el CI/CD con **más despliegues on-premise** del mundo.
- Banca, industria, defensa, telco: entornos **sin conexión a internet**.
- **Agnóstico**: GitHub, GitLab, Bitbucket, SVN, un directorio local...
- Gratis y auto-hospedado: **tú** controlas dónde corren los datos.

Y por qué a veces duele

- Hay que **mantenerlo**: actualizaciones, plugins, backups.
- Los plugins son **superpoder y deuda técnica** a la vez.
- La UI es... veterana.
- Mal configurado es un **objetivo de ataque** de primer nivel.

Controller, agentes, executors y workspace



En nuestro laboratorio hay un solo nodo: el **built-in node** (el propio contenedor de Jenkins). Suficiente para aprender; en producción **nunca** se ejecutan builds en el controller.

Vocabulario

Job / Item	Una tarea configurada
Build	Una ejecución concreta (#1, #2...)
Stage	Bloque lógico del pipeline
Step	Una acción (<code>sh</code> , <code>git</code> ...)
Workspace	Carpeta del build
Artifact	Fichero que se guarda
Executor	"Carril" de ejecución

¿Qué puedo crear en Jenkins?

Tipo	Qué es	Cuándo se usa
Freestyle project	Se configura a base de formularios en la web. Fácil de empezar, imposible de versionar.	Tareas sueltas, mantenimiento heredado
Pipeline el que usaremos	Se define en un script (el Jenkinsfile). Va en git, se revisa en un merge request y se reutiliza.	Todo lo nuevo
Multibranch Pipeline	Escanea el repo y crea un pipeline por rama automáticamente.	Equipos con muchas ramas / PRs
Folder	Carpeta para organizar y aplicar permisos por equipo.	Jenkins compartidos (como el de hoy)

Regla de oro: si la configuración de tu automatización no está en **git**, no existe. Por eso **Pipeline + Jenkinsfile**, y no Freestyle.

Plugins

Jenkins base hace muy poco. **Casi todo lo que usarás es un plugin:** hasta `git` y `pipeline` lo son.

Plugin	Para qué
<code>workflow-aggregator</code>	Pipelines declarativos (el <code>Jenkinsfile</code>)
<code>git</code>	Clonar repositorios
<code>credentials-binding</code>	Injectar secretos como variables
<code>docker-workflow</code>	Usar contenedores desde el pipeline
<code>htmlpublisher</code>	Enseñar el informe HTML de ZAP en el build
<code>timestamp</code> / <code>ansicolor</code>	Logs con hora y con color

En el laboratorio ya vienen instalados dentro de la imagen de Jenkins.

Cuidado: superficie de ataque

- Cada plugin es **código de terceros con permisos totales** sobre tu Jenkins.
- Jenkins es históricamente uno de los productos con **más CVEs** por plugins.
- Un Jenkins comprometido = **todas** tus credenciales de despliegue.

Buenas prácticas: instala solo lo necesario, revisa los avisos de seguridad (Manage Jenkins ► Security), y actualiza con ventana de mantenimiento.

Credentials: los secretos NUNCA van en el Jenkinsfile

El `Jenkinsfile` vive en **git**. Todo lo que escribas dentro, lo verá todo el que tenga acceso al repo... y quedará en el **historial para siempre**.

Tipos de credencial que usaremos

Tipo	Ejemplo de hoy
Username with password	usuario <code>root</code> + PAT de GitLab
Secret text	token de SonarQube, API key de DefectDojo
SSH private key	clonar por SSH, desplegar en un servidor
Secret file	<code>kubeconfig</code> , certificados

Cómo se usan

```
// El secreto se inyecta SÓLO en la etapa que lo necesita
withCredentials([string(
    credentialsId: 'sonar-token',
    variable: 'SONAR_TOKEN')]) {
    sh '... -Dsonar.token=$SONAR_TOKEN'
}
```

También existe `environment { X = credentials('id') }` para todo el pipeline, pero si el ID no existe **el build muere antes de la primera etapa**.

Jenkins **enmascara** el valor en la consola: verás `****` en vez del token.

Comillas simples en `sh '...'`: con `sh "...${TOKEN}"` Groovy incrusta el secreto y el enmascarado **deja de funcionar**.

BLOQUE 3

El Jenkinsfile

Pipeline as Code: tu automatización, versionada como el resto del proyecto.

Anatomía de un Jenkinsfile

```
pipeline {  
  agent any                                // DÓNDE se ejecuta  
  
  options {                                // CÓMO se comporta el job  
    timestamps()  
    timeout(time: 30, unit: 'MINUTES')  
  }  
  
  parameters {                             // QUÉ te pregunta al lanzarlo  
    booleanParam(name: 'RUN_DAST', defaultValue: false)  
  }  
  
  environment {                             // variables para TODO el pipeline  
    SONAR_HOST_URL = 'http://devsecops-sonarqube:9000'  
  }  
  
  stages {                                  // EL TRABAJO, en bloques  
    stage('Checkout') { steps { git url: '...' } }  
    stage('Analizar') { steps { sh './analiza.sh' } }  
  }  
  
  post {                                    // SIEMPRE al final, pase lo que pase  
    always { archiveArtifacts '*.json' }  
    failure { echo 'Algo ha fallado' }  
  }  
}
```

Declarativo vs Scripted

Declarativo (el que usamos): estructura fija, más legible, errores claros. **Scripted**: Groovy puro, todo poder y toda la responsabilidad.

Empieza siempre en declarativo; si necesitas lógica, mete un bloque `script { ... }` dentro.

Se llama exactamente **Jenkinsfile**, sin extensión, en la **raíz** del repositorio. Igual que `.gitlab-ci.yml` lo era para GitLab.

Los *steps* que vas a usar hoy

Step	Qué hace
<code>sh 'comando'</code>	Ejecuta shell (Linux). En Windows: <code>bat</code>
<code>echo 'texto'</code>	Escribe en el log
<code>git url:...</code>	Clona un repositorio
<code>archiveArtifacts</code>	Guarda ficheros en el build
<code>withCredentials</code>	Injecta un secreto
<code>publishHTML</code>	Publica un informe HTML
<code>script { ... }</code>	Abre un bloque con lógica
<code>error 'msg'</code>	Aborta el build en rojo
<code>unstable 'msg'</code>	Marca el build en amarillo

Variables de entorno útiles

Variable	Contenido
<code>\$WORKSPACE</code>	Ruta de la carpeta del build
<code>\$BUILD_NUMBER</code>	Nº de ejecución (1, 2, 3...)
<code>\$JOB_NAME</code>	Nombre del job
<code>\$BUILD_URL</code>	Enlace directo al build
<code>\$GIT_COMMIT</code>	Hash del commit analizado

Truco: en cualquier Jenkins, `<URL>/pipeline-syntax` te genera la sintaxis exacta de cualquier step con un formulario. Es el mejor amigo del principiante.

Verde, amarillo, rojo: el resultado es la decisión de seguridad

SUCCESS

Todo pasó. Nada que reportar.

En seguridad: ninguna herramienta encontró nada por encima del umbral.

UNSTABLE

Terminó, pero **con hallazgos**. Step: `unstable('...')`

En seguridad: "hay vulnerabilidades, míralas" — **avisa pero no bloquea**.

FAILURE

Se abortó. Step: `error('...')`

En seguridad: el **quality gate**. Nada avanza hasta que se arregle.

Traducción desde GitLab CI: lo que allí hacías con `allow_failure: true` (avisar) aquí es `unstable()` ; y `allow_failure: false` (bloquear) aquí es `error()` . **Mismo concepto, otra sintaxis.**

También existe **ABORTED** (alguien lo canceló o saltó el `timeout`) y **NOT_BUILT** (una etapa que se saltó por una condición `when`).

Disparadores

Disparador	Cómo funciona
Manual hoy	Botón <i>Build Now</i> . Perfecto para aprender.
Webhook	GitLab/GitHub avisa a Jenkins en cuanto hay push. Lo estándar en producción.
SCM Polling	Jenkins pregunta cada X minutos "¿hay algo nuevo?". Simple pero derrochador.
Cron	<code>H 2 * * *</code> — escaneos nocturnos completos.
Upstream	Se lanza cuando termina otro job.

Por qué hoy lo lanzamos a mano: con 15 personas y un solo servidor, los webhooks dispararían pipelines a la vez y se saturaría. En tu empresa, el webhook es lo normal.

Patrón real en seguridad

- **Por push:** lo rápido (secretos, SAST, SCA) — segundos o pocos minutos.
- **Cada noche:** lo lento (DAST completo, escaneo de todas las imágenes).
- **Por release:** el quality gate duro que bloquea el despliegue.

BLOQUE 4

Jenkins + Docker

Cómo un Jenkins que no tiene instalado nada consigue ejecutar Trivy, ZAP o SonarQube.

Jenkins no tiene las herramientas: las lanza en contenedores



Se llama **DooD** — *Docker out of Docker*. Le montamos a Jenkins el **socket** del demonio Docker del host, así que sus `docker run` crean contenedores **al lado** del suyo, no dentro.

La ventaja

Jenkins se mantiene **limpio**. ¿Necesitas otra versión de Trivy? Cambias una etiqueta de imagen. Nada que instalar, nada que desinstalar.

La advertencia de seguridad

Quien pueda escribir un pipeline en este Jenkins **puede hacer lo que quiera en el host**: el socket de Docker equivale a **root**.

En producción: agentes efímeros, Kubernetes, o `buildkit` / `kaniko` sin socket. En el laboratorio lo aceptamos **a propósito**, y hay que saberlo.

SI ALGÚN DÍA "EL ESCÁNER NO ENCUENTRA NADA"...

El fallo clásico: las rutas del workspace

Cuando el pipeline ejecuta:

```
docker run -v "$WORKSPACE":/src aquasec/trivy fs /src
```

...esa ruta **NO la interpreta Jenkins**. La interpreta el **demonio Docker del host**, que no sabe nada del sistema de ficheros interno del contenedor de Jenkins.

Si la ruta no existe también en el host, Docker no da error: **crea un directorio vacío** y lo monta. Resultado: el escáner analiza la nada y dice *"0 vulnerabilidades"*. **Un falso verde: lo peor que puede pasar en seguridad.**

Cómo lo resolvemos

Montamos `JENKINS_HOME` con la **misma ruta** dentro y fuera:

```
volumes:  
  - /var/jenkins_home:/var/jenkins_home
```

Así `$WORKSPACE` apunta al mismo sitio en los dos mundos.

Comprobación rápida en clase: añade un `ls -la /src` dentro del contenedor. Si sale vacío, es esto.

El mismo pipeline DevSecOps, ahora en Jenkins



GitLab CI	Jenkins	Es lo mismo
<code>.gitlab-ci.yml</code>	<code>Jenkinsfile</code>	La receta, en el repo
<code>stages:</code>	<code>stages { }</code>	Bloques del pipeline
<code>script:</code>	<code>steps { sh }</code>	Los comandos
<code>variables:</code>	<code>environment { }</code>	Variables
Settings ► CI/CD ► Variables	Credentials	Los secretos
<code>artifacts:</code>	<code>archiveArtifacts</code>	Guardar informes
<code>allow_failure: true</code>	<code>unstable()</code>	Avisar sin bloquear

La lección de fondo: aprendiste **conceptos**, no una herramienta. Cambiar de orquestador es cambiar de sintaxis, no de forma de pensar.

Ojo con las URLs: dentro de la red Docker se usan **nombres de contenedor** (`devsecops-sonarqube:9000`), nunca `localhost` .

El quality gate: cuando la seguridad tiene dientes

```
// Gitleaks devuelve 1 cuando ENCUENTRA secretos:  
// no es un error de ejecución, es un HALLAZGO.  
int rc = sh(returnStatus: true, script: 'docker run ... gitleaks  
detect ...')  
  
if (rc == 0) {  
    echo 'Sin secretos' //  
} else if (params.BLOQUEAR_SI_HAY_SECRETOS) {  
    error('QUALITY GATE: secretos detectados') // aborta  
} else {  
    unstable('Secretos detectados (aviso)') // continúa  
}
```

Fíjate en `returnStatus: true`: sin eso, `sh` aborta el pipeline en cuanto el comando devuelve algo distinto de 0 y **nunca llegarías a decidir**.

Cómo se implanta en la vida real

1. **Fase 1 — Observar:** todo en amarillo. Nadie se bloquea; se mide cuánta deuda hay.
2. **Fase 2 — Congelar:** bloquea solo lo **nuevo** (no empeorar).
3. **Fase 3 — Bloquear:** critical y secretos rompen el build, sin excepciones.

Empezar por la fase 3 el primer día es la forma más rápida de que el equipo **desactive** la seguridad.

BLOQUE 5 · 40 MINUTOS

Práctica

Tres niveles. Cada uno funciona por sí solo: si te atascas en uno, el siguiente sigue teniendo sentido.

Nivel 1 · Tu primer pipeline (10 min)

Nivel 2 · Código + escáner real (15 min)

Nivel 3 · Pipeline DevSecOps completo (15 min)

Accesos del laboratorio

Herramienta	URL (navegador)	Usuario / clave
Jenkins hoy	<code>http://localhost:8180</code>	el que creó el instructor
GitLab	<code>http://localhost:8929</code>	root / <code>DevSecOps2024!</code>
SonarQube	<code>http://localhost:9000</code>	admin / <code>DevSecOps2024!</code>
DefectDojo	<code>http://localhost:8080</code>	admin / <code>DevSecOps2024!</code>
Juice Shop	<code>http://localhost:3000</code>	(objetivo del DAST)

Si trabajas contra el servidor del aula, cambia `localhost` por su IP.

Las dos URLs de cada servicio. Tú entras por `localhost:PUERTO`. El pipeline, que corre en contenedores, usa el **nombre del contenedor**: `devsecops-sonarqube:9000`, `devsecops-defectdojo:8081`, `devsecops-juiceshop:3000`, `devsecops-gitlab:8929`.

Jenkins es compartido. Pon **tu nombre** en todo lo que crees: `pipeline-ana`, `gitlab-cred-ana` ... Así no os pisáis.

Tu primer pipeline: "Hola, Jenkins"

1 **New Item** (menú izquierdo) → nombre `pipeline-TUNOMBRE` → tipo **Pipeline** → **OK**.

2 Baja hasta la sección **Pipeline**, deja *Definition: Pipeline script* y pega:

```
pipeline {
  agent any
  stages {
    stage('Saludo') {
      steps {
        echo "Hola, soy el build #${env.BUILD_NUMBER}"
        sh 'date; whoami; pwd'
      }
    }
    stage('¿Tengo Docker?') {
      steps { sh 'docker version --format "Servidor:
{{.Server.Version}}"' }
    }
  }
}
```

3 **Save** → **Build Now** → clic en el build #1 → **Console Output**.

Qué acabas de ver

- El **Stage View**: una columna por etapa, con su tiempo.
- El **Console Output**: el log completo. **Aquí se depura todo.**
- `whoami` → `root`; `pwd` → tu **workspace**.
- Jenkins **sí** puede hablar con Docker: eso es lo que hará el trabajo pesado.

Rompe algo a propósito: cambia `sh 'date'` por `sh 'fecha'` y vuelve a lanzar. Mira cómo se pone en **rojo** y cómo el log te dice exactamente qué línea falló.

Código real + un escáner de seguridad de verdad

1 **Credencial de GitLab** — *Manage Jenkins* ► *Credentials* ► *System* ► *Global* ► *Add Credentials*: **Username with password**, usuario `root`, contraseña tu **PAT**, ID `gitlab-cred-TUNOMBRE`. 2 Job → **Configure** → sustituye el script:

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { git url: 'http://devsecops-gitlab:8929/root/devsecops-
sample.git',
                  branch: 'main', credentialsId: 'gitlab-cred-TUNOMBRE' }
    }
    stage('Secretos') {
      steps {
        sh '''docker run --rm -v "$WORKSPACE":/src -w /src \
ghcr.io/gitleaks/gitleaks:v8.18.4 detect --source /src \
--report-format json --report-path /src/gitleaks-report.json \
--redact --verbose || true'''
      }
    }
  }
  post { always { archiveArtifacts '*.json' } }
```

Fíjate en 3 cosas

- La URL usa **devsecops-gitlab**, no localhost: el job vive en la red Docker.
- El token **no está** en el script: está en **Credentials**.
- `-v "$WORKSPACE":/src` le da al contenedor de Gitleaks tu código.

Resultado: 1 hallazgo — regla `aws-access-token` en `app.py:23`.

¿Y las otras dos claves de `app.py`? Gitleaks va por **patrones conocidos**; `SuperSecret123!` no parece nada. **Falso negativo** — pero SonarQube sí lo caza. Ninguna herramienta sola basta.

El pipeline completo, desde el repositorio

1 Crea las otras dos credenciales (tipo Secret text):

ID	Valor
sonar-token	token <code>squ_...</code> de SonarQube (<i>My Account</i> ▶ <i>Security</i>)
defectdojo-api-key	API key de DefectDojo (<i>avatar</i> ▶ <i>API v2 Key</i>)

2 En tu job → **Configure** → sección Pipeline → *Definition: Pipeline script from SCM* → SCM **Git** → URL `http://devsecops-gitlab:8929/root/devsecops-sample.git` → credencial → *Branch */main* → *Script Path Jenkinsfile*.

3 **Build Now**. La 1ª vez solo registra los parámetros; lanza otra vez con **Build with Parameters**.

Esto es Pipeline as Code de verdad: Jenkins ya no guarda tu receta. La lee del repositorio en cada ejecución. Cambias el `Jenkinsfile`, haces push, y el pipeline cambia solo.

Parámetros que verás

- `RUN_SAST` — SonarQube. Si está apagado, la etapa **se omite sola** (verde)
- `RUN_DAST` — ZAP (3-5 min): **déjalo desmarcado** salvo que te sobre tiempo
- `BLOQUEAR_SI_HAY_SECRETOS` — el quality gate

Convierte el aviso en un bloqueo

- 1 Lanza el pipeline con **Build with Parameters** y marca `BLOQUEAR_SI_HAY_SECRETOS`.
- 2 Observa: el build pasa de **amarillo** a **rojo** y las etapas siguientes ni se ejecutan.
- 3 Ahora **arréglalo**: en una rama, saca el secreto de `app.py`, push, y relanza con `GIT_BRANCH=fix-secretos`.

```
# antes
AWS_SECRET = "AKIAIOSFODNN7FAKEFAKE"
# después
AWS_SECRET = os.environ.get("AWS_SECRET")
```

- 4 **Sigue en ROJO**. Gitleaks no analiza solo el código: analiza **todo el historial de git**, y el commit viejo sigue ahí.

La lección de la clase: un secreto commiteado **está quemado**. Borrarlo del fichero no lo borra de la historia, ni de los forks, ni de los backups. Lo único que sirve es **rotar la credencial**.

Si te sobra tiempo

- Activa `RUN_DAST` y mira la pestaña **Informe OWASP ZAP** del build.
- Añade `--exit-code 1` a Trivy para bloquear por CVEs críticos.
- Programa el job cada noche: *Build Triggers* ► *Build periodically* ► `H 2 * * *`.

Tres sitios, tres preguntas distintas

Jenkins :8180

¿Pasó o no pasó?

- Semáforo y tiempos por etapa
- Console Output (la verdad completa)
- Artefactos descargables

SonarQube :9000

¿Qué está mal en mi código?

- **Security Hotspots**: password en el código, MD5, CSRF off, imagen como root
- Línea exacta y cómo arreglarlo
- Verás **0 Vulnerabilities**: la detección de inyecciones (*taint analysis*) es de pago.
No está roto — es la lección.

DefectDojo :8080

¿Cuál es mi riesgo total?

- Todos los hallazgos, de todas las herramientas
- Por severidad, deduplicados
- Triage: falso positivo / riesgo aceptado

Fíjate: en DefectDojo, dentro del producto "**DevSecOps Lab**", ahora conviven engagements de **GitLab CI** (**Pipeline #...**) y de **Jenkins** (**Jenkins · ...**). Dos orquestadores, un único panel de riesgo. **Eso es gobierno de vulnerabilidades.**

BLOQUE 6

Para llevarse a casa

Comparativa honesta, seguridad del propio Jenkins y qué estudiar después.

Jenkins vs GitLab CI: ¿cuál elijo?

Criterio	Jenkins	GitLab CI
Instalación	Servidor aparte que hay que mantener	Ya viene con GitLab
Configuración	<code>Jenkinsfile</code> (Groovy) + UI	<code>.gitlab-ci.yml</code> (YAML)
Curva de aprendizaje	Más pronunciada	Más suave
Flexibilidad	Prácticamente ilimitada (+1.800 plugins)	La que da GitLab
Independencia del SCM	Total: GitHub, GitLab, SVN, Bitbucket...	Atado a GitLab
Entornos aislados / on-prem	Su terreno natural	Posible, más pesado
Mantenimiento	Tuyo: parches, plugins, backups	Integrado
Coste	Gratis (pagas en horas de administración)	Minutos de CI / licencia

En la práctica: proyecto nuevo, todo en GitLab, equipo pequeño → **GitLab CI**. Empresa con 15 años de historia, repos por todas partes, integraciones raras y sin internet → **Jenkins**. Y muchísimas empresas tienen **los dos**.

Asegurar el propio servidor de CI

Por qué es un objetivo goloso

- Guarda **todas las credenciales** de despliegue: cloud, registries, producción.
- Puede **ejecutar código arbitrario** por diseño: es su función.
- Suele tener acceso de red a **producción**.
- Con el socket de Docker montado, un pipeline malicioso = **root en el host**.

Comprometer el CI es comprometer **toda la cadena de suministro** del software.

Controles mínimos

- **Nunca** expuesto a internet sin VPN/SSO. Y con HTTPS.
- **RBAC** real (*Matrix Authorization*): no todos administradores.
- Builds en **agentes efímeros**, jamás en el controller.
- Plugins **mínimos** y actualizados; revisar los avisos de seguridad.
- Secretos en **Credentials** (o Vault), nunca en el Jenkinsfile.
- **Backup** de `JENKINS_HOME` y auditoría de quién cambia jobs.

Nuestro laboratorio incumple varias de estas a propósito (Jenkins como root, socket montado, todos administradores). Está bien saberlo: **un lab no es producción**.

Las 8 ideas que deberías recordar mañana

- 1 **Jenkins es un orquestador:** no analiza nada, **llama** a quien analiza.
- 2 **Pipeline as Code:** la receta vive en el repo, en un `Jenkinsfile`.
- 3 **Controller + agentes + executors:** quién manda, quién ejecuta y cuántos a la vez.
- 4 **Credentials:** los secretos, fuera del código y enmascarados en el log.
- 5 **Contenedores como herramientas:** Jenkins limpio, versiones controladas.
- 6 **Verde / amarillo / rojo** es una **decisión de seguridad**, no un color.
- 7 **Quality gate progresivo:** primero medir, después congelar, al final bloquear.
- 8 **El CI es un activo crítico:** protégelo como protegerías producción.

Y la más importante: lo que aprendiste con GitLab CI te sirvió intacto aquí. Las herramientas cambian cada 3 años; **los conceptos, no.**

¿Preguntas?

Para seguir practicando

- Monta un **Multibranch Pipeline** y mira cómo aparece un job por rama.
- Configura un **webhook** de GitLab para que el push lance Jenkins solo.
- Añade una etapa de **tests** con `junit` y mira las gráficas.
- Sustituye `docker run` por `agent { docker { image '...' } }`.

Recursos

- `<tu-jenkins>/pipeline-syntax` — generador de sintaxis
- `jenkins.io/doc/book/pipeline` — la referencia oficial
- `docs/jenkins/Practica-Jenkins-Alumno.pdf` — el paso a paso de hoy
- `sample-project/Jenkinsfile` — el pipeline completo comentado